

Reinforcement-Learning-Based Traffic Signal Control System

Final Project Report

Aydan Karmali, Jenish Paudel, Samreet Johal

cs4470Y Software Maintenance & Configuration Management

Department of Computer Science

Western University

April 6th, 2026

Prof. Fang (Fiona) Fang, Department of Electrical and Computer Engineering

Prof. Nazim Madhavji, Department of Computer Science

Glossary of Terms and Definitions

TraCI (Traffic Control Interface): A Python interface for controlling a running **SUMO** simulation. It allows external code to read traffic states and modify signals, routes, or parameters in real time via a socket connection.

SUMO (Simulation of Urban Mobility): A traffic simulator that models individual vehicle movement using car-following, lane-change, and intersection rules. Networks and traffic demand are defined in configuration files.

SMDP (Semi-Markov Decision Process): A decision framework where actions last for variable durations. Unlike standard MDPs, decisions occur only when certain conditions are met, allowing flexible signal timing.

Double DQN: A reinforcement learning algorithm that reduces overestimation in DQN by separating action selection and evaluation across two networks.

ϵ -greedy: An exploration strategy where a random action is chosen with probability ϵ ; otherwise, the best estimated action is taken. ϵ typically decreases during training.

Epoch: One complete pass through all training episodes.

Episode: A single simulation run of one traffic scenario.

ThroughputCompositeReward : A reward combining **throughput**, **queue length**, and **waiting time** to evaluate intersection performance. Coefficients α , β , γ control the contribution of each term.

Structured Abstract

- **[Context and motivation]** Urban traffic signal control is often governed by fixed-time or actuated rules that cannot adapt well to changing traffic demand throughout the day. Such systems can increase delay, queue length, congestion, and inefficient use of intersection capacity.
- **[Question/problem]** Given a historical traffic dataset of an intersection, what is the most optimal method of sequencing the traffic light?
- **[Principal ideas and solution]** The solution we built is an end-to-end ML pipeline that accepts a historical dataset of an intersection and the intersection's configuration, trains a reinforcement learning model, and generates the most optimal traffic light sequences for that intersection.
- **[Technology and implementation]** The system is implemented in Python using SUMO for simulation for traffic simulation alongside PyTorch library for setting up the reinforcement learning algorithms and Streamlit for the user interface. It supports data upload, configuration settings & generation, model training, baseline evaluation, results logging, and downloadable output artifacts.
- **[Impact]** Traffic signal inefficiency contributes directly to urban congestion, emissions, and wasted commute time. This platform addresses that by automatically generating optimal signal timing schedules from nothing more than an intersection definition and vehicle count data. A reinforcement learning model is trained on those inputs and produces the most effective signal sequence for that intersection, making what was once a complex research task straightforward and accessible.
- **[Limitations]** The reported results are simulation-based and limited to a single signalized intersection. The system does not support multi-intersection coordination, and the final learned policy should be interpreted within the constraints of the available training budget and evaluation setup.

Table of Content

1. Introduction	1
2 Background and Related Work	2
2.1 Application context	2
2.2. System context	2
2.3. Context Diagram	3
2.4 Related work	3
2.5 Positioning	3
3 Concepts, Equations, etc.	4
4 Project Goal and Objectives	4
5 System requirements	5
5.1 Functional Requirements	5
5.2 Quality Requirements	8
6 Development Strategy and Implementation	9
Development Framework(s) and Libraries	9
Development Tools Used	9
Programming Languages Used	9
Noteworthy Points About Implementation	9
Development Platform and Operational Environment	10
Development Methodology	10
Other (Limitations)	10
7. Solution Created (Result)	10
7.1 System Architecture and Design	10
7.3 Operational Evidence	13
7.4 System Validation	15
7.5 Experimental Results	15
8 Discussion	15
8.1 Impact of the system	16
8.2 Limitations of the system	16
9 Conclusions	16
10 Lessons Learnt	17
11 Acknowledgements	18
13 Appendix	20

1. Introduction

Traffic signal control remains an important problem in urban transportation because the way an intersection is timed directly affects congestion, vehicle delay, queue formation, and the overall efficiency of traffic movement. Traditional fixed-time signal plans are simple to implement, but they do not adapt well to changing traffic conditions across different times of day. Even actuated approaches, while more responsive than fixed schedules, still rely on predefined control logic rather than learning from repeated traffic patterns. Because of this, reinforcement learning has become an attractive approach for traffic signal optimization in simulation-based research environments.

This project addresses the problem of how to generate more effective traffic-light control for a signalized intersection using historical traffic demand data. More specifically, the project investigates whether a reinforcement-learning-based controller can be trained in SUMO to produce signal decisions that outperform conventional fixed-time and actuated baselines under the same simulated conditions. Rather than treating the work as only a machine learning experiment, the project focuses on building a complete software system that accepts traffic data and intersection settings, prepares simulator inputs, trains and evaluates a controller, and presents usable outputs to the user.

The motivation for the project is both practical and technical. From a practical perspective, inefficient traffic signal timing contributes to unnecessary congestion, wasted travel time, and poor use of available intersection capacity. From a technical perspective, traffic control provides a strong setting for reinforcement learning because an agent can repeatedly observe traffic conditions, take actions, and improve through feedback in simulation. This project therefore aims to show that reinforcement learning can be used not just as a theoretical idea, but as part of a reproducible end-to-end platform for single-intersection traffic signal optimization.

The solution created is a reinforcement-learning-based traffic signal control system implemented in Python using SUMO, TraCI, PyTorch, and Streamlit. The system supports data ingestion, configuration generation, route and network preparation, model training, baseline evaluation, logging, plotting, and export of a learned timing schedule in `schedule.json` form. The final implementation includes both a command-line workflow and a browser-based front end, making the system more accessible for experimentation and demonstration. In evaluation, the learned Double DQN controller was compared against fixed-time and actuated baselines, with results reported using the common simulation environment and shared held-out test conditions where appropriate.

The significance of this project lies in its practical integration of preprocessing, simulation, learning, evaluation, and usability features into one coherent workflow. Its contribution is not a claim of new reinforcement learning theory, but a working software platform that makes controlled traffic-signal experimentation easier to run, reproduce, and interpret. By generating logs, comparison plots, checkpoints, and a readable learned timing schedule, the system provides outputs that are useful for analysis and future extension. At the same time, the project remains scoped to simulation and a single intersection, so its conclusions should be interpreted within those limits.

The remainder of this report is organized as follows. Section 2 presents the background of the project, the system context, and related work. Section 3 introduces key concepts and equations needed to understand the reinforcement-learning approach. Section 4 states the overall project goal and objectives, while Section 5 defines the system’s functional and quality requirements. Section 6 describes the development strategy and implementation environment. Section 7 presents the solution created, including the architecture, testing, operational evidence, validation, and experimental results. Section 8 discusses the system’s impact and limitations, Section 9 gives the conclusions, Section 10 summarizes lessons learned, and the appendices provide supporting diagrams and experimental details.

2 Background and Related Work

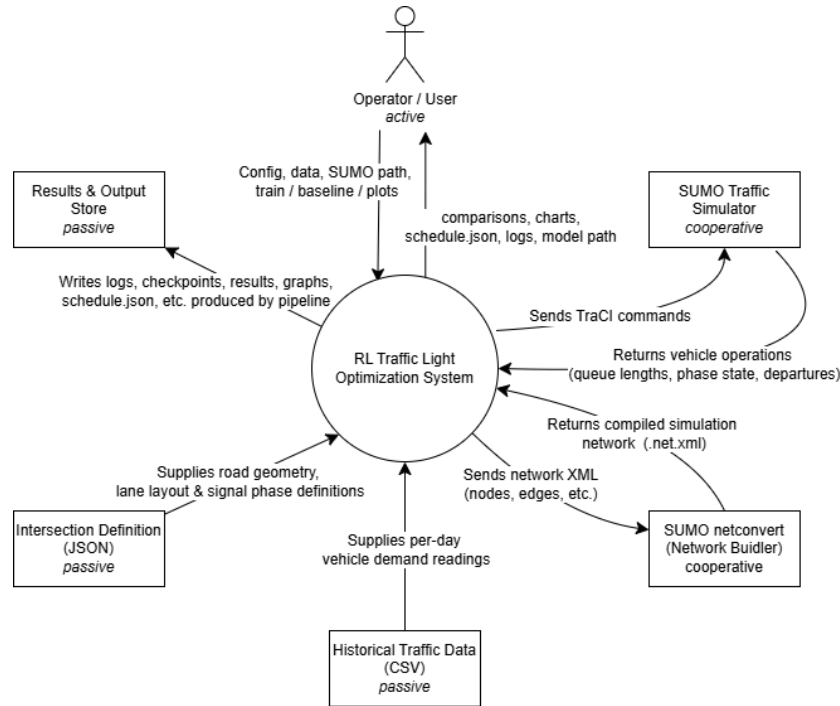
2.1 Application context

- Traffic signal control is a well-established challenge in urban transportation, where signal timing has a direct impact on vehicle delay, queue length, and overall traffic flow. While many real-world systems employ a range of strategies from fixed schedules to more advanced adaptive controls, the complexity of optimizing signal timing across varying traffic conditions continues to be an active area of research. This project explores reinforcement learning as an approach to signal timing optimization at a simulated single intersection, contributing a platform that learns timing sequences from intersection definitions and vehicle count data.

2.2. System context

- At its core, the platform accepts traffic demand data and an intersection definition as inputs, and works through a structured pipeline to produce an optimized signal timing schedule. The data is first transformed into simulation-ready artifacts compatible with Simulation of Urban Mobility (SUMO), which then runs the traffic environment the reinforcement learning agent trains within. Once training is complete, the learned controller is tested against baseline signal policies to evaluate its performance.

2.3. Context Diagram



[Figure 1 Context Diagram]

- The operator interacts primarily through the Streamlit web application, which accepts traffic data and settings, starts RL training and baseline evaluation against SUMO, and presents model–baseline comparisons, reward-focused comparison plots, tabular outputs, and downloads the schedule. JSON, charts, and paths to the run directory where KPI summaries (results.csv) and logs are stored. The same system can be driven from the command line for scripted or reproducible runs.

2.4 Related work

- SUMO is widely used in intelligent transportation research because it enables detailed vehicle-level simulation and supports external control through TraCI (Lopez et al., 2018). Reinforcement learning has been extensively studied for adaptive signal control, where an agent learns from repeated interaction with a traffic environment rather than relying on manually designed rules. Deep Q-learning remains a practical approach when the action space is discrete and the state is represented numerically (Haydari & Pourmohammad, 2022).

2.5 Positioning

- This project is an applied software and experimentation effort rather than a claim of novel RL theory. Its contribution is a working end-to-end pipeline that combines preprocessing, simulation, training, baseline comparison, and usability into one coherent workflow.

3 Concepts, Equations, etc.

Semi-Markov decision process (SMDP). The traffic signal control problem is modelled as a semi-Markov process because decisions are made at variable time intervals rather than every identical simulator tick. In this project, the controller decides whether to hold or advance the current traffic-light phase at configured decision points, while yellow and all-red transitions are forced between decisions.

Observation/state. Before each decision, the agent receives a numerical representation of the current traffic condition. This includes features derived from the simulation state: queue lengths, current phase, information, elapsed phase times, and time-of-day context. These values form the state input used by the RewardPolicy to estimate action reward.

Action space. The controller operates over a discrete action space. At each decision point, it chooses among traffic-signal actions, such as holding the current phase or advancing to the next allowed phase, subject to signal timing constraints. Because the actions are discrete, value-based reinforcement learning is a suitable approach.

Double DQN. Double Deep Q-Network (Double DQN) is a value-based reinforcement learning method that reduces overestimation bias by separating action selection from target-value evaluation. In this project, we use the online network to select the next action and the target network to evaluate it during training updates. This improves learning stability compared with a basic DQN formulation.

ϵ -greedy exploration. During training, the policy uses ϵ -greedy exploration: with probability ϵ , it chooses a random action, and otherwise, it selects the action with the highest predicted Q-value. This balances exploration of alternative signal decisions with exploitation of what the model has already learned.

Reward function. Training uses ThroughputCompositeReward: at each decision instant, the

reward is
$$R = \gamma \frac{\Delta V}{n_{dep}} - \alpha \bar{Q} - \beta \sigma_Q$$
 where ΔV counts vehicles newly observed on departure lanes since the last decision (accumulated each simulation step), $\bar{Q}$ is the mean halting count over TLS-controlled lanes, σ_Q is the spread of those lane queues, and n_{dep} is the number of departure lanes when normalization is on. Default weights are $\gamma = 1.0$, $\alpha = 0.4$, and $\beta = 0.15$, trading off throughput against total delay and cross-lane unfairness. Essentially, the reward says get vehicles through the intersection, don't let queues blow up overall, and don't let one approach sit in a huge line while others are fine.

Timing constraints. Signal actions are constrained by realistic traffic-control rules, including minimum green, maximum green, yellow clearance, and all-red intervals. These constraints prevent unrealistic rapid switching and ensure that the learned controller operates within safe and interpreted signal behaviour limits.

4 Project Goal and Objectives

The overall goal of this project was to design and implement a reinforcement learning-based system for optimizing traffic signals at a simulated single intersection in SUMO, not only a learning model, but a complete software pipeline capable of taking traffic demand data, preparing simulation inputs, training a control policy, evaluating it against baseline methods, and presenting results through both command-line and front-end workflows. We achieved this goal by creating an integrated experimental platform that combines preprocessing, simulation, learning, evaluation, and usability features into a single system.

The project objectives were as follows:

- **O1.** Give the agent a real feedback loop: it watches what is happening at the intersection in SUMO, picks signal actions, and gets better the more simulated days it lives through.
- **O2.** Translate demand from spreadsheet counts to SUMO-compatible routes, networks, and train-test splits, and conclude each serious run with saved logs and checkpoints for later reproduction or review.
- **O3.** Evaluate the learned controller against fixed-time and actuated baselines using the same simulator, network, timing constraints, and reward logging, and report comparisons on overlapping held-out test days where the train/test split assigns different day IDs to the RL test set and the baseline batch (see Appendix B).
- **O4.** Improve accessibility through a Streamlit-based interface supporting data upload, configuration generation, training execution, and results inspection.
- **O5.** Produce a reusable and configurable platform that supports further tuning and experimentation.

5 System requirements

This section defines the key functional requirements (FR) and quality requirements (QR) for the final system. Several of these requirements are adapted from Progress Report 2, especially the demand-generation, network-validation, and RL-control requirements, but they have been revised to reflect the final integrated software platform rather than the earlier subsystem-only view.

5.1 Functional Requirements

ID	Requirement	Rationale
FR1	The system shall accept traffic demand data in CSV, XLS, or XLSX form and use it to generate simulation-ready demand for SUMO.	Keeps the system usable with realistic traffic-count inputs and preserves a requirement already established in PR2.
FR1.1	The system shall validate that required traffic columns and time fields are present before route generation begins.	Prevents silent failures and invalid simulations caused by missing or malformed input data.

FR1.2	The system shall support multi-day datasets and generate non-overlapping simulation timelines.	Needed for train/test splitting and repeatable experimentation across multiple traffic days.
FR2	The system shall generate SUMO-compatible route/flow artifacts from the input demand data.	The RL controller cannot be trained unless valid demand artifacts are produced.
FR2.1	The system shall generate per-day route or flow files suitable for repeated training and evaluation runs.	Supports the final pipeline's day-based training and testing design.
FR2.2	The system shall fail clearly when required movement-to-edge mappings are incomplete or invalid.	Preserves the PR2 fail-fast behaviour and improves debugging and reproducibility.
FR3	The system shall generate or validate a SUMO-compatible road network for the target intersection.	The simulation environment depends on a valid network definition.
FR3.1	The system shall produce network artefacts that load successfully in SUMO without fatal route or network errors.	Ensures the generated simulation inputs are operational, not just syntactically present.
FR3.2	The system shall generate supporting mapping information, such as edge mapping, needed to connect traffic movements to valid network edges.	Preserves the network-aware mapping requirement from PR2 and supports route generation correctness.
FR4	The system shall implement a reinforcement-learning-based traffic signal controller for training and evaluation in SUMO	This is the core product goal of the project.
FR4.1	The controller shall observe traffic state features derived from the current simulation state before making each decision.	The learned controller must act on traffic conditions rather than fixed timing alone.
FR4.2	The controller shall choose among discrete traffic-signal actions and apply them subject to timing constraints.	Reflects the actual discrete phase-control design used in the codebase.
FR4.3	The system shall compute a scalar reward at each decision interval to support policy learning.	Required for reinforcement learning to function.
FR4.4	The system shall support experimentation with various reward definitions, including	Preserves evidence from development that multiple reward strategies were explored.

	wait-time, delta wait-time, throughput-only, and composite rewards.	
FR5	The learning subsystem shall support experience replay, target-network synchronization, and repeated policy updates over training episodes.	Retains key RL design requirements identified in PR2 and matches the implemented DQN-style architecture.
FR5.1	The system shall store transitions collected during simulation and sample them during training.	Required for stable DQN-style learning.
FR6	The system shall enforce traffic-signal timing constraints, including minimum and maximum green durations.	This was explicitly identified in PR2 and remains essential for realistic and safe signal behaviour.
FR6.1	The controller shall not switch a phase before the minimum green duration has been satisfied.	Prevents unrealistic oscillation.
FR6.2	The controller shall be punished for exceeding the maximum green duration limit.	Allows the agent to adapt to high-volume traffic through exceeding the maximum duration while still keeping it constrained through punishments.
FR7	The system shall evaluate the learned controller against at least two baseline controllers.	Prevents the RL solution from being assessed in isolation.
FR7.1	The system shall provide a fixed-time baseline using the same simulation and evaluation pipeline.	Gives a simple non-adaptive reference.
FR7.2	The system shall provide an actuated baseline using the same simulation and evaluation pipeline.	Gives a stronger rule-based comparison.
FR8	The system shall produce persistent output artifacts from training and evaluation runs.	Needed for reproducibility, analysis, and reporting.
FR8.1	The system shall save training logs, test logs, checkpoints, and final model artifacts.	Supports traceability of experiments.
FR8.2	The system shall generate comparison plots from saved logs.	Supports interpretation of results in the final report.

FR8.3	The system shall generate a schedule.json timing summary derived from evaluation by aggregating learned phase durations across time-of-day buckets.	Preserves an interpretable timing artifact for analysis and reporting.
FR8.4	When a timing summary is available, the front end shall display the learned signal timing schedule and allow the user to download the generated schedule.json file.	Makes the learned schedule easy for the user to view and download.
FR9	The system shall provide both a command-line workflow and a front-end workflow for running experiments.	Reflects the final system, which includes both main.py and Streamlit-based execution.
FR9.1	The front end shall allow the user to upload traffic data and generate or edit required configuration content.	Matches the implemented Streamlit behaviour.
FR9.2	The front end shall allow the user to launch training and baseline evaluation runs.	Makes the system easier to use and demonstrate.
FR9.3	The front end shall display run feedback such as logs, charts, or summary outputs.	Improves usability and transparency during execution.

5.2 Quality Requirements

ID	Requirement	Rationale
QR1	The system shall be reproducible, with configuration-driven execution and saved run artifacts.	Important for experimentation, debugging, and reporting.
QR2	The system shall be modular, separating preprocessing, environment interaction, observation, reward, policy, replay buffer, evaluation, and UI concerns.	Makes the system easier to maintain and extend.
QR3	The system shall provide clear failure behaviour when inputs, mappings, or generated artifacts are invalid.	Adapted directly from PR2's fail-fast requirements.
QR4	The system shall be usable by someone who is not required to modify source code for every run.	Justifies the Streamlit front end and configuration-based workflow.

QR5	The system shall support comparative evaluation fairness, meaning the RL controller and baseline controllers use the same evaluation environment and timing constraints.	Ensures comparisons are meaningful and defensible in the report.
QR6	The system shall support extensibility for additional rewards, policies, and experiments.	Important because the project evolved through reward testing and grid-search style experimentation.
QR7	The system shall remain desktop-operable using locally installed SUMO and Python dependencies.	Reflects the actual deployment model of the final system.
QR8	The system shall produce outputs that are sufficiently interpretable for human analysis, including logs and visual summaries.	Needed for evaluation, debugging, and the final capstone report.

6 Development Strategy and Implementation

- **Development Framework(s) and Libraries**

PyTorch is used to implement the value-based reinforcement learning policy, while SUMO provides the traffic model. TraCI is used to exchange commands and state between Python and the simulator. Streamlit supplies a browser-based interface for uploading traffic counts, generating or editing JSON configuration files, launching training or baseline runs, and viewing streamed logs. Supporting libraries such as Matplotlib, NumPy, and pandas are used for preprocessing, input/output operations, and plotting. SUMO, PyTorch, Streamlit, and the broader Python ecosystem are open-source foundations, while project-specific functionality is implemented in custom preprocessing, the agent loop, reward definitions, and experimental code.

- **Development Tools Used**

The project uses Git for version control and pip for dependency management, with a root-level requirements.txt file. Configuration is divided between main.py (which defines paths, training parameters, simulation settings, and component selection) and JSON files such as columns.json, intersection.json, reward_configuration.json, and policy_configuration.json. Automated tests are located in the Testcases/ directory and focus on route and network generation. Batch experimentation is supported through experiments/grid_search/. Each full run produces a timestamped directory under logs/, containing checkpoints and JSON logs.

- **Programming Languages Used**

The system is implemented in Python.

- **Noteworthy Points About Implementation**

The main.py file orchestrates the full workflow, including input validation, route and dataset generation, network construction, component assembly, and execution of training

through the Trainer. The reinforcement learning system is modular, with interchangeable components such as the environment wrapper, observation builder, reward function, policy, replay buffer, and optional scheduler. These components can be swapped by modifying class bindings in `main.py`. Fixed-time and actuated baselines, along with evaluation and plotting scripts, are included to enable direct comparison under identical demand conditions. Furthermore, Runs invoke `modelling/schedule_export.py` to build `schedule.json`; `streamlit_app.py` surfaces it under Learned Signal Timing Schedule with Download `schedule.json`.

- **Development Platform and Operational Environment**

The system is designed for a local desktop environment, where Python is executed from the repository root and SUMO runs on the same machine. No cloud infrastructure is required. Users configure `SUMO_HOME` or rely on default paths documented in the code (e.g., for Windows installations). Training is initiated using `python main.py`, with an optional `--gui` flag to enable the SUMO graphical interface. The Streamlit application runs in a browser and writes temporary files under `streamlit_runs/` before invoking the same pipeline as the CLI. Outputs, including checkpoints, trained models, and logs, are stored locally under `logs/` and related `src/data/` directories. SUMO is the primary external dependency, and while an NVIDIA GPU with CUDA can accelerate PyTorch, execution time is typically dominated by simulation steps.

- **Development Methodology**

Development followed an iterative approach. Initial work focused on establishing a functional reinforcement learning pipeline, including preprocessing, environment interaction via TraCI, observation design, reward formulation, and training. Subsequent development introduced baseline controllers, visualization tools, a Streamlit interface, and grid-search experimentation over reward and policy configurations. This progression reduced integration risk before expanding usability and experimental capability.

- **Use of Datasets**

We started from the Canadian Urban Data Centre catalogue, Traffic Volumes—Multimodal Intersection Turning Movement Counts (City of Toronto): short-term turning-movement counts with 15-minute bins in the raw exports. That real count series was used as a base. We then generated synthetic multi-day demand from it so the simulator had enough distinct days for training and for a held-out test split. The synthetic table is still consumed as a CSV (or spreadsheet), mapped with `columns.json`, tied to `intersection.json`, and passed through BuildRoute to generate per-day SUMO flows and `split.json`.

- **Other (Limitations)**

The Streamlit interface executes training within the same process as the server, which simplifies demonstrations but is less suitable for long-running experiments. The system is limited to single-intersection simulation and does not support real-world deployment or coordinated multi-intersection traffic control.

7. Solution Created (Result)

7.1 System Architecture and Design

7.1.1 Component-Connector (See Appendix Fig. 2 for Diagram)

The system is organized into four layers. The Input and User Interaction layer ingests traffic demand CSVs and JSON configuration files, with users interacting via a Streamlit web UI (uploading, auto-generating configs, launching runs) or the CLI (`main.py`, `evaluate_baseline.py`). The Pipeline Orchestration layer, coordinated by `main.py`, invokes `BuildRoute.py` to generate per-day SUMO route files and `split.json` and `BuildNetwork.py` to construct the network using `netconvert`, while `evaluate_baseline.py` runs fixed-time and actuated policies in parallel on the same test days. The Learning and Simulation Core features a trainer iterating over training days and epochs; in each episode, the agent gathers observations (queue lengths, phase, phase timer, and time of day), selects actions via a Double DQN policy with epsilon-greedy exploration, applies phase changes in the SUMO environment through TraCI, receives a `ThroughputCompositeReward`, and stores transitions in a Replay Buffer for gradient updates. The External Services and Outputs layer saves model checkpoints (`final_model.pt`), JSON training/testing logs, aggregated learned phase schedules (`schedule.json`), baseline comparison logs, and plots in timestamped run directories. [See Appendix Fig. 2 for Diagram.]

Architectural patterns: layered pipeline; strategy/plugin component binding via JSON; SumoEnvironment wrapper isolating TraCI; DQN tactics including uniform replay, target network sync, epsilon decay, and cosine LR scheduling.

7.1.2 Design Models

The use case model has a single actor—the User/Analyst—performing five tasks: Prepare Input Data, Configure Simulation, Train RL Model, Evaluate Performance, and View Plots and Results. The full functional decomposition diagram and CLI sequence diagram are provided in Appendix A.

[See Appendix Figure 3 for Use Case diagram.]

7.1.3 Key System Qualities and Rationale

Quality	Design Treatment	Rationale
Interoperability	SUMO-standard <code>net.xml</code> / <code>rou.xml</code> ; TraCI	Compatible with external tools
Modifiability	Swappable components + JSON configs	Easy experimentation
Reproducibility	<code>split.json</code> + config logging	Controlled experiments
Performance	Simulation-bound; PyTorch	Efficient training
Portability	Python + SUMO	Runs on standard systems
Usability	Streamlit UI	Easier demos

Quality	Design Treatment	Rationale
Interoperability	SUMO-standard net.xml / rou.xml; TraCI	Compatible with external tools
Modifiability	Swappable components + JSON configs	Easy experimentation
Reproducibility	split.json + config logging	Controlled experiments
Reliability	Validation + checkpoints	Fail-safe execution
Security/privacy	Local execution	No external exposure

7.2 System Testing

Testing covered unit, subsystem, integration, and regression levels using Python's unittest framework, with all tests writing to isolated temporary directories and leaving src/ unchanged.

7.2.1 Unit Testing focused on preprocessing: Test_BuildRoute.py (6 classes) validates configuration loading, CSV parsing, day assignment, per-day CSV and .rou.xml outputs, and split reproducibility, while Test_BuildNetwork.py (9 classes) verifies phase generation, all four SUMO XML files, lane counting, cross-file node/edge consistency, and proper failure when netconvert is unavailable.

7.2.2 Subsystem Testing ensured BuildRoute.py and BuildNetwork.py generate complete outputs on valid inputs and fail clearly on malformed ones; all nine reward classes were validated for correct instantiation and computation, and reward_configuration.json merging produced expected resolved parameters.

7.2.3 Integration and Regression Testing verified end-to-end execution via python main.py, confirming generation of train_log.json, pretest_log.json, test_log.json, final_model.pt, and checkpoints, with both test suites re-run after major preprocessing, reward, or pipeline changes to detect regressions.

7.2.4 Selected Test Cases

Test ID	Level	What Was Tested	Expected Result
TC-01	Unit	TestSplitJson—train/test reproducibility with fixed seed	Identical train and test day lists across two independent runs using seed 42
TC-02	Unit	TestSumoFlowFiles—non-overlapping flow intervals per route	No two flow elements on the same route share overlapping begin/end times
TC-03	Unit	TestWriteTll—signal state string validity	All phase state strings contain only valid SUMO characters (G, g, y, r, R) and are of consistent length
TC-04	Unit	TestEdgeCases—graceful failure when netconvert absent	FileNotFoundError raised cleanly; no crash or silent failure

TC-05	Unit	TestAllFilesWrittenTogether - cross-file node and edge consistency	All edge IDs in .con.xml exist in .edg.xml; all node IDs in .edg.xml exist in .nod.xml
TC-06	Integration	Full CLI run via python main.py	train_log.json, test_log.json, pretest_log.json, and final_model.pt present in the run directory upon completion

TC-01–02 supports FR1/FR2; TC-03–05 supports FR3/QR3; TC-06 supports FR4, FR5, FR7, FR8.

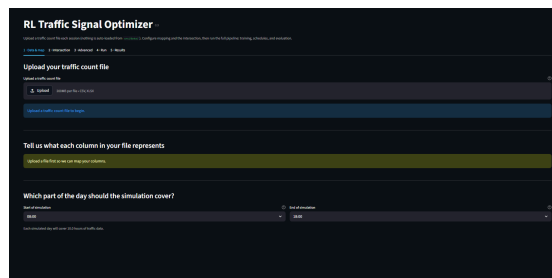
7.2.5 Convergence testing:

The model was run multiple times with varying parameters and functional edits to ensure the model was actually learning.

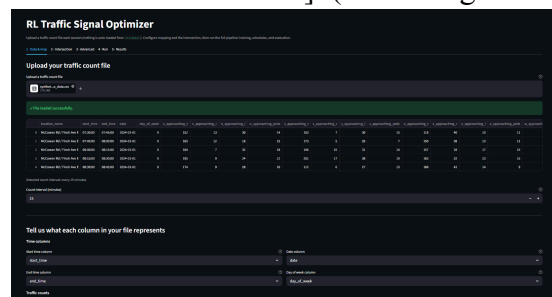
A grid search over reward functions and hyperparameters was conducted to select the best configurations; a summary of findings is provided in Appendix B.

7.3 Operational Evidence

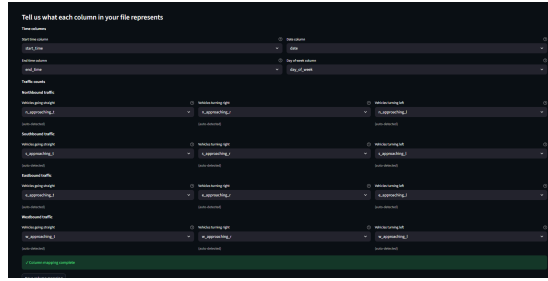
The system was verified across three execution scenarios: Scenario A—CLI: python main.py generates console logs, a timestamped src/data/results/ directory, JSON logs, and final_model.pt; Scenario B—Streamlit: streamlit run streamlit_app.py provides live progress, a scrolling log, real-time reward charts, and produces the same artifacts as the CLI, with Figures 2–9 illustrating each workflow stage, and upon completion the Results section displays train, pretest, and test logs and charts, along with a Learned Signal Timing Schedule (median phase durations per 15-minute bucket from test episodes) and a downloadable schedule.json, which also appears when loading prior runs via Load Existing Run Results; Scenario C—Grid Search: python experiments/grid_search/gridsearch.py outputs results.csv and leaderboard.csv summarizing all experimental runs.



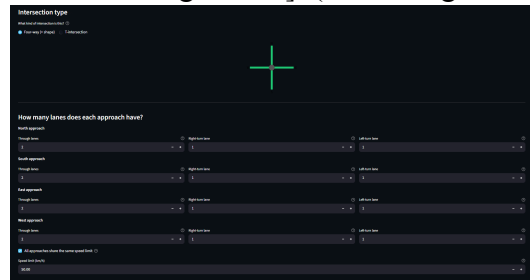
[Figure 5 Full interface on launch] (click images for full screen)



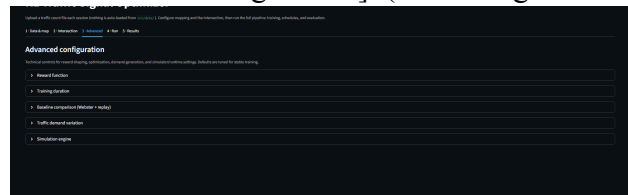
[Figure 6 CSV upload and data preview] (click images for full screen)



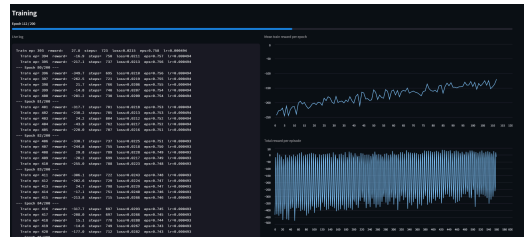
[Figure 7 Column configuration] (click images for full screen)



[Figure 8 Intersection Configuration] (click images for full screen)



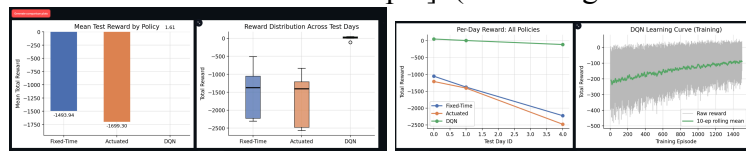
[Figure 9 Advanced Training Configuration] (click images for full screen)



[Figure 10 Training run in progress] (click images for full screen)

Results Comparison [ThroughputCompositeReward]			
Day	DQN	Fixed-Time	Actual
0	50.82	-1053.01	-1209.91
1	6.23	-1375.62	-1405.78
2	N/A	-2305.60	-2568.97
4	-112.58	-2224.94	-2478.92
5	N/A	-510.54	-832.94
6	19.58	N/A	N/A
9	44.00	N/A	N/A
Mean	1.61	-1493.94	-1699.38

[Figure 11 Baseline evaluation output] (click images for full screen)



[Figure 12 Comparison Plots]

Day	Phase	Time	Duration	Priority	Order
0	A_Landis	0	00:00	10	1
1	A_Landis	1	00:00	40	1
2	A_Landis	2	00:00	10	1
3	A_Landis	3	00:00	10	1
4	A_Landis	4	00:00	40	1
5	A_Landis	5	00:00	10	1
6	A_Landis	6	00:00	10	1
7	A_Landis	7	00:00	40	1
8	A_Landis	8	00:00	10	1
9	A_Landis	9	00:00	40	1

Download schedule.json

Artifacts: C:\Users\jenish\Documents\TrafficSim\artifacts\schedule.json

[Figure 13 **Learned Signal Timing Schedule** panel and **Download schedule.json** button.]
(click images to see them in full screen)

7.4 System Validation

Single-intersection offline optimization scenario: The system can successfully support a traffic analyst who wants to optimize one known intersection using historical traffic-count data and structured configuration files. In this setting, the platform can generate SUMO inputs, train a reinforcement-learning controller, evaluate it against baselines, and produce interpretable outputs such as plots, logs, and schedule.json.

Controlled controller-comparison scenario: The system can support fair comparison between the learned controller and fixed-time/actuated baselines because all are evaluated in the same simulation environment with the same network and timing constraints. Comparative conclusions are strongest when reported only on overlapping held-out test days.

Usability scenario: The Streamlit front end allows users to upload data, configure settings, launch runs, and inspect outputs without editing the source code for every run. However, the system still expects technically prepared inputs and some familiarity with traffic-simulation concepts.

Reproducibility scenario: The system supports repeatable experimentation through saved logs, checkpoints, configuration-driven execution, and persistent run artifacts. This makes it suitable for offline research and controlled evaluation.

Real-world deployment scenario: The system does not yet cope with direct deployment to real traffic infrastructure, since it has only been validated in simulation and does not integrate with live sensors or signal hardware.

Multi-intersection scenario: The current implementation does not support coordinated control across multiple intersections, so validation is limited to a single-intersection setting.

7.5 Experimental Results

The Double DQN agent was trained for 300 epochs across five traffic days (1,500 training episodes) using ThroughputCompositeReward, a cosine learning-rate schedule ($5 \times 10^{-4} \rightarrow 1 \times 10^{-4}$), and ϵ -greedy exploration from 1.0 toward 0.05. At the end of training, $\epsilon \approx 0.492$, so exploration had not fully decayed, and the policy should be interpreted as partially converged within the allotted budget.

On the five held-out test days used for the final agent, greedy evaluation produced a mean cumulative reward of +1.61 per episode, compared with -1,172.75 for the untrained random policy on the same five days (+100.1% improvement in mean reward).

For comparison to classical controllers, Table B1 (Appendix B) reports mean reward on the three test days shared by both the DQN evaluation and the fixed-time/actuated batch (days 0, 1, and 4): -18.51 (DQN) versus -1,551.19 (fixed-time) and -1,698.20 (actuated)—approximately 98.8% and 98.9% higher mean reward on those days. DQN was also evaluated on two additional test

days (6 and 9) that were not part of the baseline batch; those rows appear in Table B1 but are not averaged against baselines.

On the three overlapping days alone, the mean reward improved from -1838.64 (random policy) to -18.51 (learned policy).

Across the five DQN test episodes, mean completed throughput was about 951 vehicles per 2-hour evaluation window, with a mean crossing rate of about 13.2% (as logged in test_log.json). The learned plan was exported as schedule.json.

8 Discussion

8.1 Impact of the system

- The project shows that a reinforcement-learning signal controller trained in simulation can match or outperform fixed-time and actuated control on held-out days, supporting RL as a practical option for single-intersection optimization.
- A key contribution is the integrated, reproducible pipeline: preprocessing, SUMO network/flow generation, training, baseline comparison, and reporting are all in one repository. This reduces setup friction for researchers who want to run controlled RL traffic experiments without deep tool-specific configuration.
- Usability is improved through Streamlit, where users can upload traffic counts, generate configuration files, and launch runs without editing source code. The schedule.json export and UI preview further translate learned behaviour into a stakeholder-readable timing artifact.
- The modular architecture (reward, policy, observation, environment components) supports extension beyond one experiment and makes reward/policy iteration straightforward. Evaluation is kept consistent by using the same SUMO network, controller constraints, and reward framing across compared controllers; because archived baseline and final DQN runs use partially different test IDs, headline comparisons are reported on overlapping days (0, 1, 4), while pre-train vs post-train RL comparisons use identical day sets (Table B1, Appendix B).

8.2 Limitations of the system

- The current scope is a single simulated intersection and has not been validated with live sensor streams or deployed in operations, so external validity is limited. Streamlit training runs also block the app process, making CLI execution preferable for long jobs unless background execution is added.
- Porting to new intersections still requires manual preparation of intersection.json and columns.json (lane counts, phases, and mappings), and the fixed simulation window with synthetic/historical demand cannot fully capture real-world variability (incidents, weather, and atypical pedestrian surges).
- The agent also did not fully converge within the 300-epoch budget ($\epsilon = 0.492$), but the trajectory is encouraging: performance improved from random baseline (-1172.75) to partially trained policy ($+1.61$) on the same test days, indicating learned adaptive behaviour. Longer runs or adjusted epsilon decay is, therefore, the most direct next step.

9 Conclusions

Urban traffic congestion at signalized intersections is often worsened by fixed-time control strategies that cannot respond effectively to changing demand. The goal of this project was to design and implement a reinforcement-learning-based traffic signal control system for a simulated single intersection, supported by a complete software pipeline rather than only a standalone learning model. This included preprocessing traffic-count data, generating SUMO-compatible simulation artifacts, training and evaluating a controller, comparing it against baseline methods, and presenting outputs through both command-line and front-end workflows.

That goal was substantially achieved. A Double Deep Q-Network agent operating in a Semi-Markov Decision Process setting was trained and evaluated within the SUMO microscopic traffic simulator using traffic demand derived from the City of Toronto turning-movement dataset. In the final evaluation, the learned policy improved the mean cumulative reward from $-1,172.75$ for the untrained random policy to $+1.61$ on the same five held-out test days. On the three test days shared by both the learned agent and the classical baselines, the DQN controller achieved a mean reward of -18.51 compared with $-1,551.19$ for fixed-time control and $-1,698.20$ for actuated control. Although training had not fully converged within the 300-epoch budget, the results still showed clear evidence of learned adaptive behaviour under the reported evaluation setup.

The project also produced several technical artifacts that support reuse and future extension. These include the modular reinforcement learning pipeline and system architecture described in Section 7.1, the testing and execution workflows described in Sections 7.2 and 7.3, the experimental evaluation reported in Section 7.5, the Streamlit web interface for end-to-end experiment management, and the `schedule.json` export that converts learned behaviour into a readable timing plan. Together, these artifacts make the project more than a proof-of-concept model: they form a reproducible experimentation platform for single-intersection traffic signal control studies.

Overall, this work shows that reinforcement learning can be integrated into a practical and reproducible traffic-signal experimentation system and can outperform simpler controllers in simulation under the tested conditions. At the same time, the conclusions must be interpreted within the project's scope: the study is limited to a single simulated intersection, requires manual preparation for new intersections, and does not yet support real-world deployment. The most direct next steps are longer training runs, improved transferability to new intersections, and broader validation under more varied traffic conditions.

10 Lessons Learnt

- **Reinforcement learning is not just a model choice; it changes how the whole system must be designed.** We learned that building an RL-based traffic controller required much more than implementing a neural network. The project depended on a full loop of environment design, observation selection, reward design, action constraints, logging, and evaluation. In practice, a large part of the work was making sure the agent was learning from the right signals in a stable and measurable way.
- **Safety and realism matter as much as optimization performance in real-world-inspired control problems.** A traffic signal controller cannot be judged only

by whether it improves reward or throughput. Real signal systems must respect constraints such as minimum green time, maximum green time, yellow clearance, and all-red intervals, and these constraints strongly shape what kinds of decisions are even acceptable. This showed us that in practical software systems, especially those connected to transportation or public infrastructure, correctness and safety constraints are as important as intelligence.

- **Hyperparameter tuning and reward design were far more time-consuming than expected.** One of the biggest lessons from this project was that achieving useful RL behaviour depends heavily on careful tuning. Small changes to learning rate, exploration decay, reward weights, or training duration could noticeably affect results, and many runs were needed before the system behaved in a meaningful way. This made it clear that reinforcement learning projects often require significant experimentation time, and that progress is rarely linear.
- **A working result does not automatically mean a fully converged or final solution.** We learned that an RL agent can show clear improvement over baselines even while still being only partially trained. This was important because it forced us to think critically about what our results actually meant. Rather than claiming the controller was fully optimized, we had to interpret the outcome honestly and distinguish between evidence of learning and proof of a finished solution.
- **Reproducibility and tooling save a huge amount of time in experimental projects.** Because tuning and evaluation required many repeated runs, features such as saved logs, checkpoints, configuration files, and consistent train/test splits became essential. Without those pieces, it would have been much harder to compare reward functions, debug failures, or explain results in the final report. This taught us that good experiment management is not just convenience; it is part of the technical quality of the system.
- **Usability matters even in technically heavy projects.** Adding the Streamlit interface helped us see the value of making a complex system easier to operate and demonstrate. A system can be technically strong but still difficult to use if everything depends on manual code edits and command-line setup. Making the workflow more accessible improved the project itself and made the final output feel more like a usable software platform than only an academic prototype.

11 Acknowledgements

We would like to express our sincere gratitude to our supervisor, Professor Fang (Fiona) Fang, for her guidance, support, and valuable feedback throughout this work. We also thank PhD student Guan Qiang for their helpful insights and assistance. Additionally, we appreciate the support of the course coordinator, Professor Nazim H. Madhavji, for facilitating this course.

References

- Haydari, A., & Pourmohammad, S. (2022). Deep reinforcement learning for traffic signal control: A review. *IEEE Transactions on Intelligent Transportation Systems*, 23(11), 18979–18995. <https://doi.org/10.1109/TITS.2022.3195534>
- Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y., Hilbrich, R., Lücken, J., Rummel, J., Wagner, P., & Wissner, E. (2018). Microscopic traffic simulation using SUMO. In 2018 21st International Conference on Intelligent Transportation Systems (ITSC) (pp. 2575–2582). IEEE. <https://doi.org/10.1109/ITSC.2018.8569938>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>
- City of Toronto. (n.d.). Traffic volumes at intersections for all modes. Canadian Urban Data Catalogue. Retrieved April 6, 2026, from <https://data.urbandatacentre.ca/catalogue/city-toronto-traffic-volumes-at-intersections-for-all-modes>
- PyTorch. (n.d.). PyTorch documentation. Retrieved April 6, 2026, from <https://docs.pytorch.org/docs/stable/index.html>
- Eclipse SUMO. (n.d.). Simulation of Urban MObility—Documentation. Retrieved April 6, 2026, from <https://sumo.dlr.de/docs/>
- Verma, P., Singh, G., & Sharma, P. (2025). Traffic light timing optimization using Webster’s formula: Graphical representation. *International Journal of Scientific Research & Engineering Trends*, 11(3).

13 Appendix

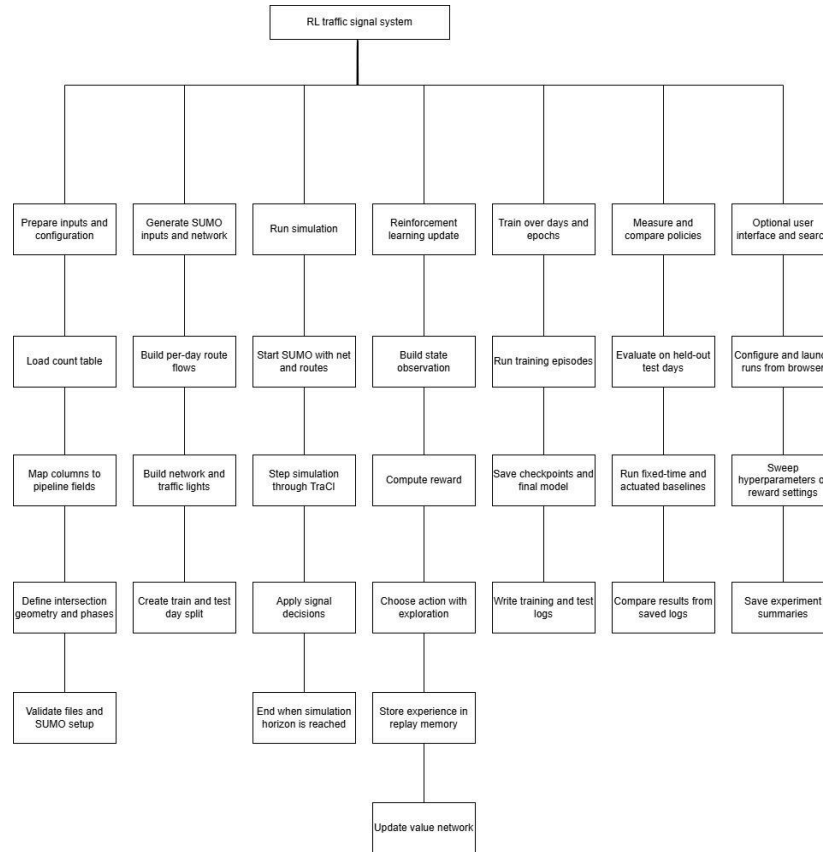


Figure A1—Functional Decomposition of the RL Traffic Signal System. The system decomposes into seven parallel functional branches, each refined top-to-bottom into implementation-level steps.

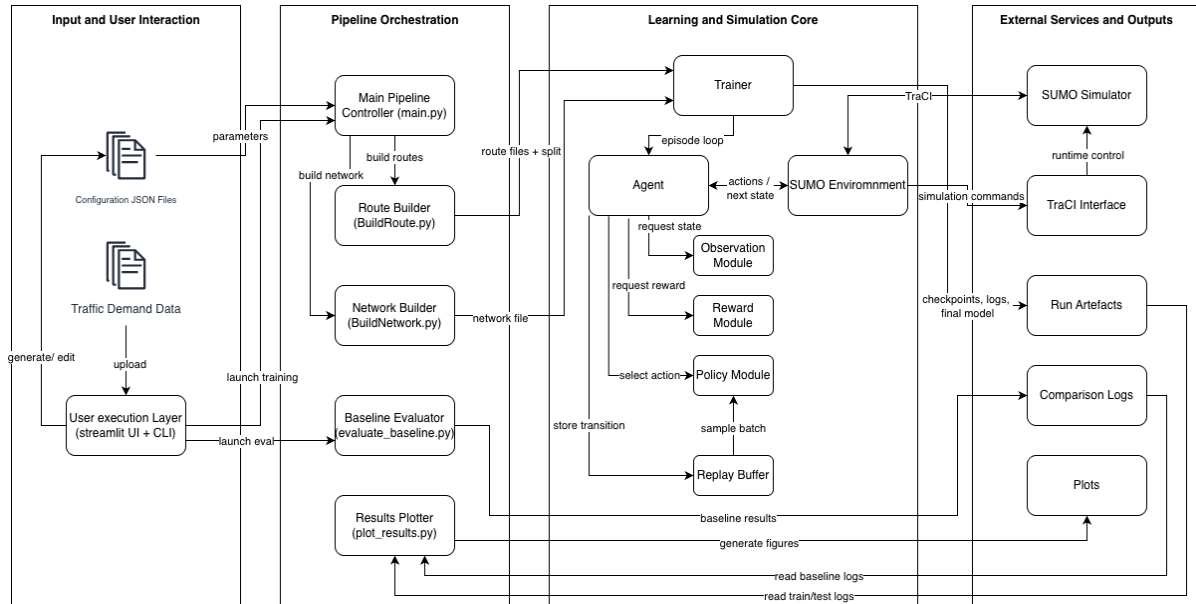
(Same 2-hour simulation window: 28,800 s – 36,000 s. DQN tested in greedy mode.)

Test Day	DQN (post-train)	DQN (pre-train, random)	Fixed-Timed	Actuated
Day 0	+50.82	−991.47	−1053.01	−1209.91
Day 1	+6.23	−1408.79	−1375.62	−1405.78
Day 4	−112.58	−3115.65	−2224.94	−2478.92
Mean (Days 0,1,4)	−18.51	−1172.75†	−1551.19	−1698.20

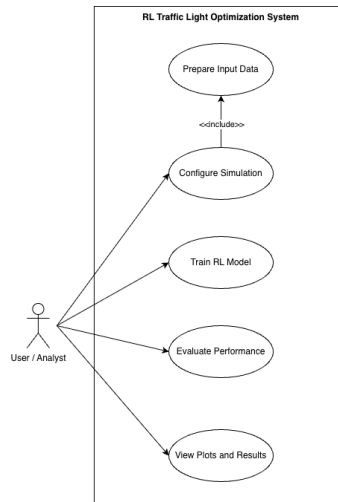
[Table B]

† Baselines were evaluated on the original split (Days 0, 1, 2, 4, 5); Days 6 and 9 were not included in the baseline evaluation. The three common days (0, 1, 4) are used for the direct performance comparison.

Reward function: ThroughputCompositeReward ($\alpha=0.4$, $\beta=0.15$, $\gamma=1.0$, normalized). Higher is better.



[Figure 2 Component-Connector Diagram]



[Figure 3: Use Case Diagram]

This project's use cases centre on a traffic analyst or researcher who prepares a scenario (traffic data and settings), configures training, runs the reinforcement-learning pipeline to learn signal timings, optionally compares the learned policy to baselines, and reviews outputs such as logs and metrics.